Desk Number _____

Student Number _____

Student Name _____

School of Science and Engineering

MIDTERM EXAMINATION

Term 2, 2025

CSC1001 Introduction to Computer Science

Examination Duration: 120 minutes

Reading Time: 10 minutes

This examination has __5__ questions.

The score distribution of the 5 question is Question 1(40%), Question 2(20%), Question 3(10%), Question 4(15%), Question 5(15%)

Exam Conditions:

This is a FORMAL Examination

This is a RESTRICTED OPEN BOOK Exam. Maximum of one (1) sheet of handwritten notes double-sided are permitted

Materials Permitted In The Exam Venue:

Maximum of one (1) sheet of handwritten notes double sided are permitted. **NO OTHER MATERIALS PERMITTED**

Calculators without the functionalities of programming and file storage are permitted.

Materials To Be Supplied To Students:

1 × 6 Page Answer Booklet

You must write all your answers in your answer book.

Question 1. (40%)

Please describe what will be printed for each program as below.

	Program	Printed Results

1) 3%	`x = (2**3 % 5) // 2 + float("77".replace("7", "1"))`
	`print(x)`
	
	Explanation: 2**3 % 5 is 3, and 3 // 2 is 1. Replacing each 7 in "77" with 1 gives "11", or 11.0; the sum is 12.0.
		Answer: 12.0

2) 3%	`s1, s2, s3 = eval("15-3"), str(15-3), "15-3"`
	`print(s1 + len(s2) + len(s3))`
	
	Explanation: eval("15-3") gives 12; str(15-3) is "12"; s3 is "15-3". Their string lengths are 2 and 4, so 12 + 2 + 4 = 18.
		Answer: 18

3) 3%	`t = [10, 20, "python", False, 80]`
	`print(t[2][2:5].upper(), t[0:2])`
	
	Explanation: t[2] is "python" and [2:5] selects "tho", whose uppercase form is "THO". t[0:2] selects [10, 20].
		Answer: THO [10, 20]

4) 3%	`x = 10`
	`y = []`
	
	`if (not x > 5) or (y and x % 2 == 0):`
	`  print("csc1001")`
	`elif len(y) == 0 and x > 8:`
	`  print("hello")`
	`else:`
	`  print("world")`
	
	Explanation: x is 10 and y is empty. The first condition is false; len(y)==0 and x>8 are both true, so the elif branch prints hello.
		Answer: hello

5) 3%	`n = 12`
	`i = 1`
	`while i < n:`
	`  print(i, end="*")`
	`  i = i * 2`
	
	Explanation: i starts at 1 and doubles after each print. The values 1, 2, 4, and 8 are below 12; 16 ends the loop.
		Answer: 1*2*4*8*

6) 4%	`def log_event(event, cache=[], max_size=1):`
	`  cache.append(event)`
	`  if len(cache) > max_size:`
	`  cache[:] = cache[1:]`
	`  print(cache)`
	
	`log_event("start")`
	`log_event("finish", max_size=2)`
	
	Explanation: Appending finish makes the cache length 2, greater than max_size 1, so the oldest event is removed.
		Answer: ["finish"]

7) 4%	`data = "student_ID_12345.csv"`
	`course = ""`
	`recording = False`
	`for char in data:`
	`  if char == '_':`
	`  recording = not recording`
	`  continue`
	`  if char == '.':`
	`  break`
	`  if recording:`
	`  course += char`
	`  print(course)`
	
	Explanation: The two underscores toggle recording on and then off. Digits follow the second underscore, so none are collected before the period ends the loop; course remains empty.
		Answer: "" (empty string)

8) 3%	`for i in range(3):`
	`  if i == 2:`
	`  print(i, end=",")`
	`  break`
	`  else:`
	`  print("Loop finished")`
	
	Explanation: For i=0 and i=1 the if condition is false, so the else suite prints Loop finished twice. At i=2 the if branch prints 2, then break ends the loop.
		Answer: "Loop finishedLoop finished2,"

9) 3%	`s = "user-tongxin-li-admin@example.com"`
	`print(s.split("@")[0].split("-")[-1])`
	
	Explanation: Splitting at @ selects the username. Splitting on hyphens and taking the final piece returns admin.
		Answer: "admin"

10) 4%	`counts = {'a': 5, 'b': 10}`
	`counts['c'] = counts.get('c', 0) + 1`
	`print(counts['a'], counts['c'])`
	`print('b' in counts)`
	
	Explanation: The missing count for c defaults to 0 and is incremented to 1. The outputs are a count 5, c count 1, and True for b membership.
		Answer: 5 1 True

11) 3%	`h = open('example.txt', 'w')`
	`h.write('Hello')`
	`h.close()`
	`h = open('example.txt', 'r')`
	`s = h.read()`
	`print('First read:', s)`
	`s = h.read()`
	`print('Second read:', s)`
	
	Explanation: The first read consumes Hello. The next read starts at end of file and returns an empty string.
		Answer: "First read: Hello" then "Second read: "

12) 4%	`d = {'b':1, 'a':2, 'c':0}`
	`lst = []`
	`for k, v in d.items():`
	`  if v > 0:`
	`  lst.append(k)`
	`  lst.sort()`
	`  print(lst)`
	
	Explanation: Only b and a have positive values. Sorting those keys alphabetically gives [a, b].
		Answer: ["a", "b"]

Question 2. (5*4% = 20%)

Please answer the following short questions.

1) Provide the Binary and Hexadecimal representations of the decimal number 29.25.

Answer: Binary = 11101.01; hexadecimal = 1D.4.

Explanation: 29 is 11101_2 and 0.25 is $.01_2$. Grouping into four-bit digits gives 0001 1101 . 0100 = $1D.4_{16}$.

2) Please list two advantages of using binary numbers in modern computer systems.

Answer: Binary states are reliable in electronic circuits and support simple, efficient Boolean logic.

Explanation: Two voltage states are easy to distinguish despite electrical noise, and Boolean operations map directly to digital circuits.

3) Please describe the differences between arguments and parameters.

Answer: Parameters are names in a function definition; arguments are values passed in a function call.

Explanation: In def f(x), x is a parameter; in f(3), 3 is an argument.

4) Will the following be implemented without causing an error? (Yes/No)

```
data = ("apple", 5, "orange")
print(data[0])
data[1] = 10
print(data)
```

Answer: No. The code causes a TypeError.

Explanation: Tuples are immutable. Reading data[0] is valid, but assigning a new value to data[1] raises TypeError.

Explain the difference between a list and a tuple in Python. Provide one reason to use a tuple instead of a list.

Answer: A list is mutable; a tuple is immutable. A tuple is useful for fixed data that should not change.

Explanation: Lists allow item assignment, while tuple elements cannot be reassigned after creation.

5) What is the purpose of the return keyword in a function? What happens if a function does not have a return statement?

Answer: return sends a result to the caller and ends the function. Without an explicit return, the function returns None.

Explanation: The caller can use the returned value. Reaching the end without returning a value produces None.

Question 3. (10%)

Write a Python program that plays a number-guessing game with a user. The game should follow these rules:

- The computer fixes a **secret random integer** between 1 and 50.
- The user gets a maximum of **5** attempts to guess the number.
- After each guess, the program must provide feedback to the user, telling them if their guess was "**Too high**!" or "**Too low!**".
- If the user guesses correctly, the game ends, and the program prints a success message, like "**Correct! You guessed it in [N] attempts.**" where N is the total number of attempts.
- If the user runs out of attempts, the game ends, and the program should reveal the secret number, printing "**Sorry, you've run out of attempts. The secret number was [X].**" where X is the correct integer.

Answer and explanation:

```
import random
secret = random.randint(1, 50)
for attempt in range(1, 6):
    guess = int(input("Guess a number from 1 to 50: "))
    if guess == secret:
        print(f"Correct! You guessed it in {attempt} attempts.")
        break
    elif guess > secret:
        print("Too high!")
    else:
        print("Too low!")
else:
    print(f"Sorry, you've run out of attempts. The secret number was {secret}.")
```

randint(1, 50) includes both endpoints. The loop allows at most five guesses. It gives high/low feedback, breaks on success, and runs its else clause only if all tries finish without a break.

Question 4. (15%)

Please first explain the usage of the built-in string method **find()**. Then, write a function **myFind**(*main_string*, *sub_string*):

- *main_string* is the string to be searched.
- *sub_string* is the string to search for.
- The function should return the **ending index** of the first occurrence of *sub_string* within *main_string*.
- If *sub_string* is not found, the function should return -1.

You may not use the built-in find() or index() methods in your implementation.

(e.g., if *main_string* is "hello world csc1001" and *sub_string* is "world", the result should be 10)

Answer and explanation: The built-in find(sub_string) returns the starting index of the first match, or -1 if absent. The example specifies the final matched character's index: world occupies indices 6 through 10, so it returns 10.

```
def myFind(main_string, sub_string):
  if sub_string == "":
    return -1
  for i in range(len(main_string) - len(sub_string) + 1):
    j = 0
    while j < len(sub_string) and main_string[i + j] == sub_string[j]:
      j += 1
    if j == len(sub_string):
      return i + len(sub_string) - 1
  return -1
```

Each possible starting position is checked character by character. A full match returns the index of its last character, matching the example; if none matches, the function returns -1.

Question 5. (15%)

You are in charge of counting the votes for a class election. You have a file, **votes.txt**, where every line contains the name of the candidate who received a vote.

Example **votes.txt** content:
Alice
Bob
Alice
Charlie
Alice
Bob
...

Please write a Python program that:

- Reads the **votes.txt** file.
- Counts the total number of votes for each candidate.
- Creates a new file named **results.txt** that lists each candidate and their total vote count, sorted alphabetically by the candidate's name.

Answer and explanation:

```
votes = {}
with open("votes.txt", "r") as source:
  for line in source:
    name = line.strip()
    if name:
      votes[name] = votes.get(name, 0) + 1
with open("results.txt", "w") as output:
  for name in sorted(votes):
    output.write(f"{name} {votes[name]}\n")
```

strip() removes each line ending. The dictionary tracks vote totals, and sorted(votes) writes candidates alphabetically. The sample results are Alice 3, Bob 2, and Charlie 1.

END OF EXAMINATION